

Wall Projections

Interactive Exhibitions for Bristol Museums

By Sonny, Sergi, Thomas, Daniel & Miłosz

Our client and vision

Our client was Bristol Museums, who wanted a reusable system allowing users to learn relevant information by interacting with 3D printed artefacts on a table.

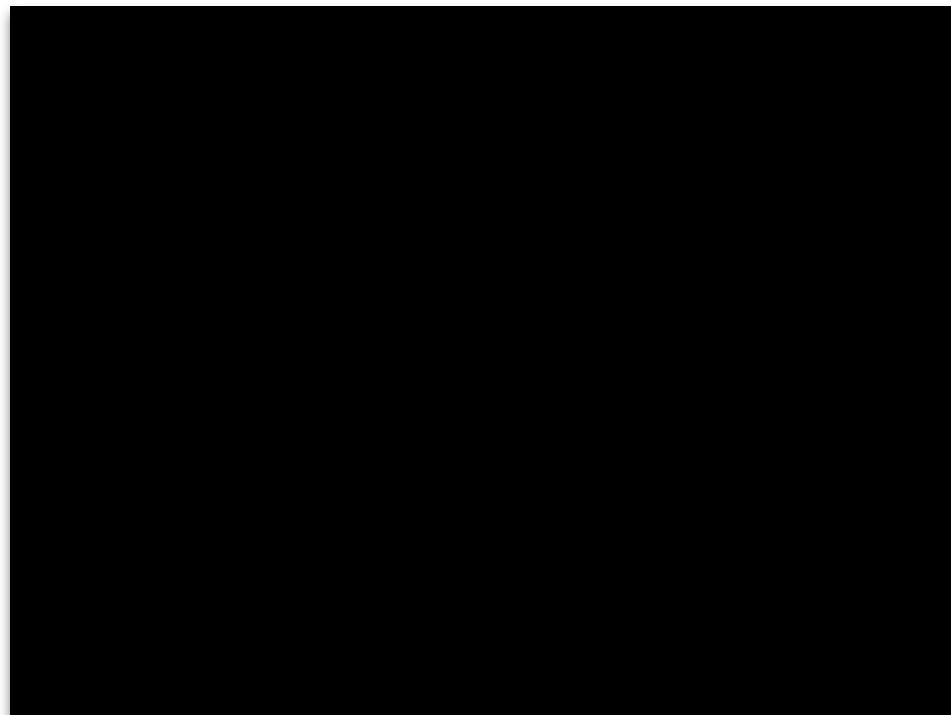
This would work with two projectors, one projecting “hotspots” on the 3D printed artifacts, and the other projecting the information onto a nearby wall.



Some User Stories

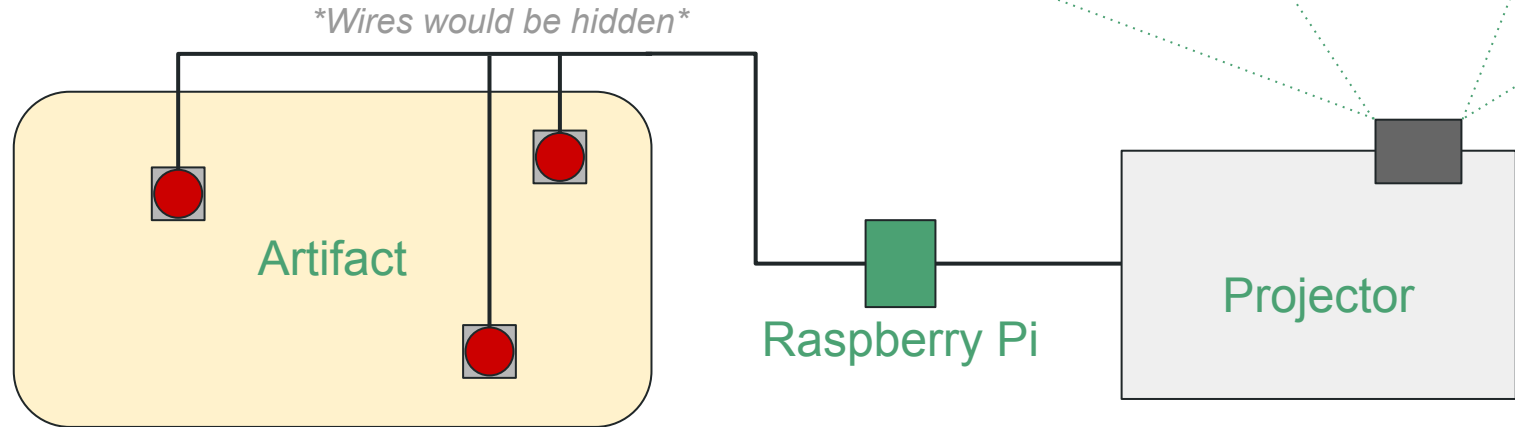
- As a **museum visitor**, I want to interact with the installation, so that I can learn about the artefacts in an engaging way.
- As an **exhibit designer**, I want to edit hotspot information with a graphical interface, so that it is easier to line up the hotspots with the artefacts.
- As an **exhibit designer**, I want to edit the location of hotspots with a graphical interface, so that it is intuitive and takes less time.
- As an **exhibit designer**, I want to package the data, so that the museum curator can load it up without manual editing.

Demo



Minimal Viable Product

An artifact is set up with buttons placed at points of interest. When a user presses a button, information related to the point of interest is displayed on the Projector.



Frontend development - MVP

- Technology exploration stage
 - JavaFX? Other Java UI frameworks?
Raspberry Pi? Cross-compilation? .NET?
- Initial proof-of-concept design
 - Focused on a working prototype with core functionalities
 - Text
 - Images
 - Videos
- Manual configuration setup
 - The configuration file had to be written by hand

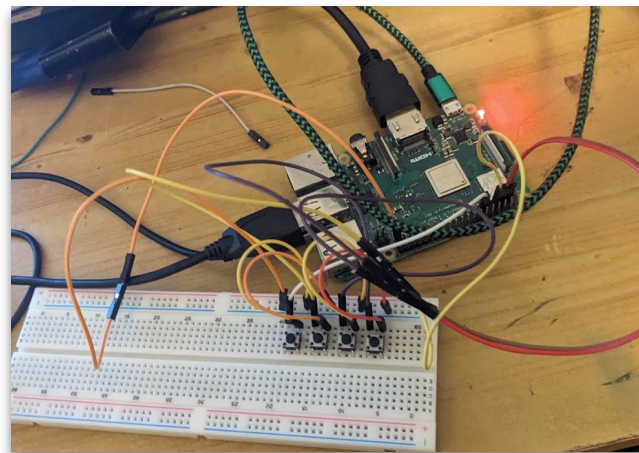
Dr Richard Smith Junior and the Horwood Book

The story behind the book is one of controversy and Dr Richard Smith's fascination for collecting macabre death memorabilia. Dr Richard Smith Junior Dr Richard Smith (1772-1843) was a surgeon at Bristol Infirmary from 1796 until his death. Amongst other things, he is known for his anatomisation of executed criminals. In April 1802 he dissected [...]



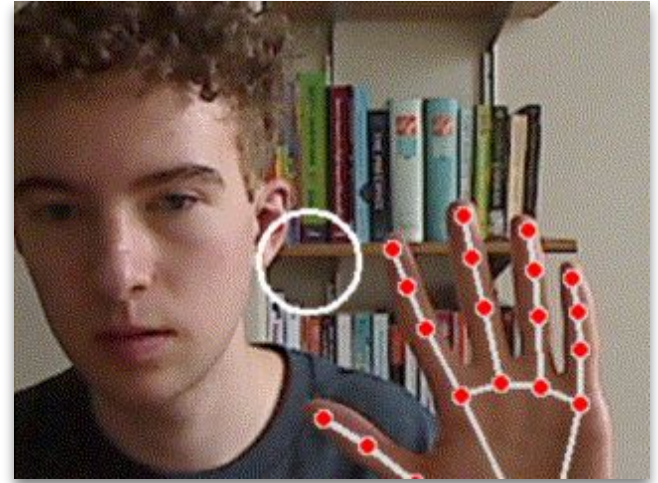
Backend development - MVP

- MVP
 - Raspberry Pi button handling
 - Rudimentary C# ↔ Python interop
- Exploration stage
 - Computer vision? ML? Lasers? Touch sensors?
Light occlusion? Multiple cameras?
 - Narrowed down to:
MediaPipe, sensing decrease in light



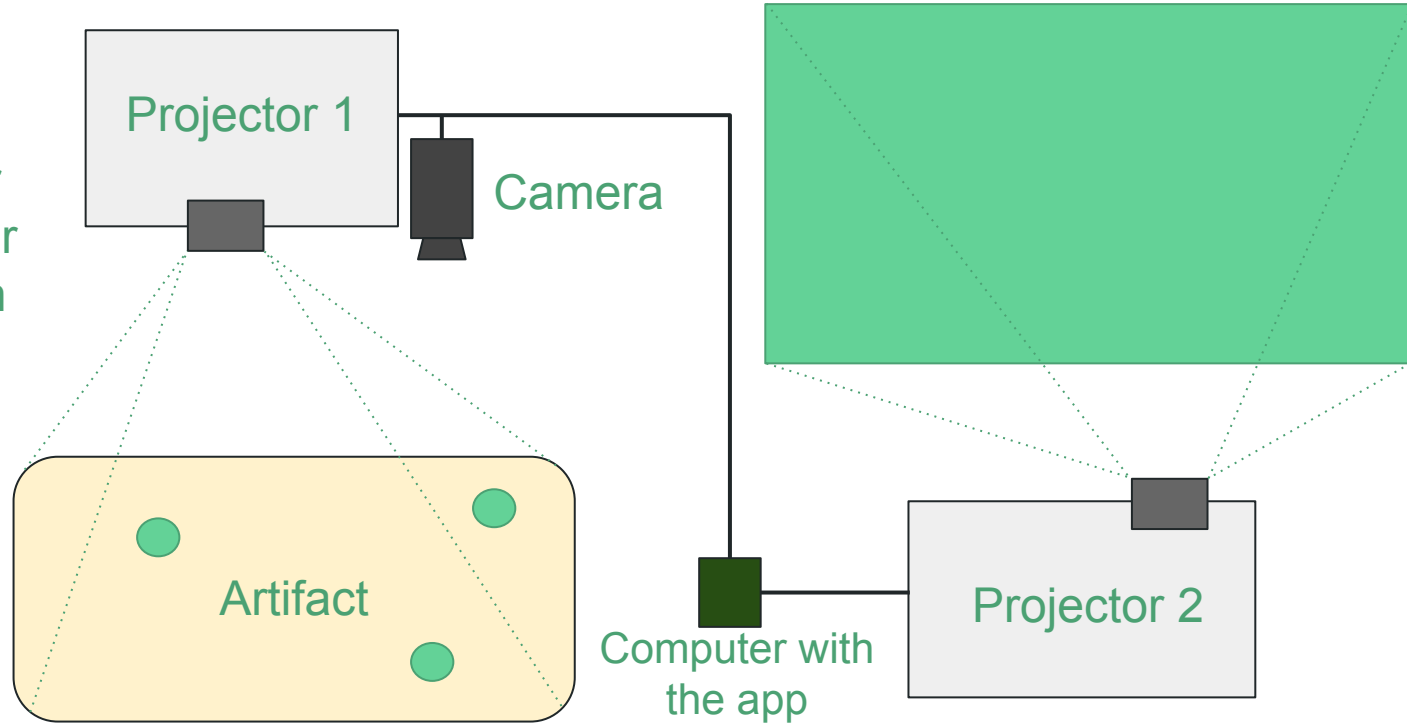
Backend development - MVP

- Proof-of-concept using MediaPipe
 - Hand landmarking
 - Rough idea of scope and components of the project



Beta & Final release Setup

An artifact is set up with hotspots projected at points of interest. When a user places their hand over a hotspot, information related to the hotspot is displayed on Projector 2.

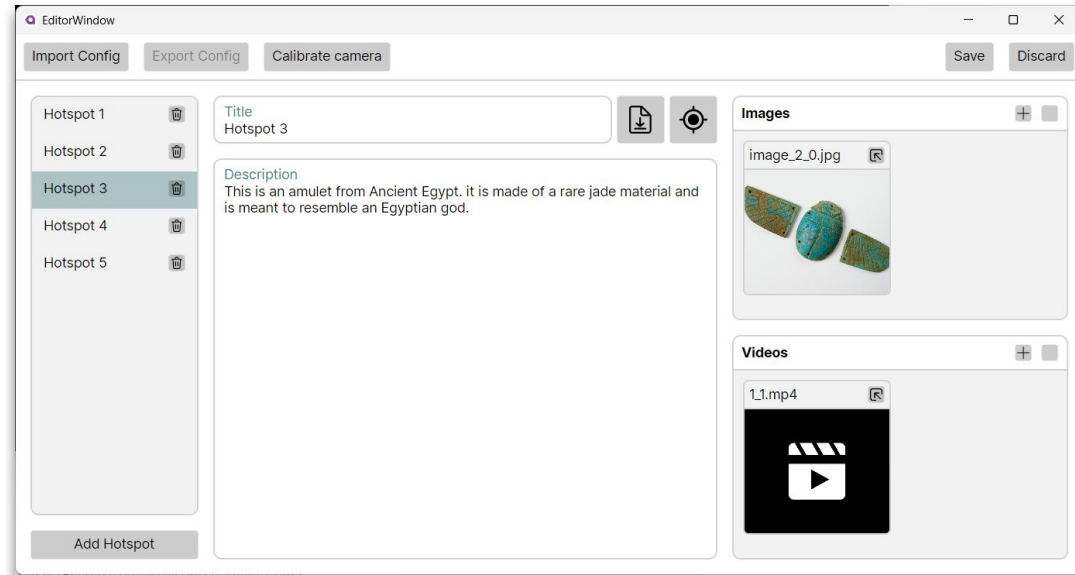


Frontend development - Beta release

Configuration Editor

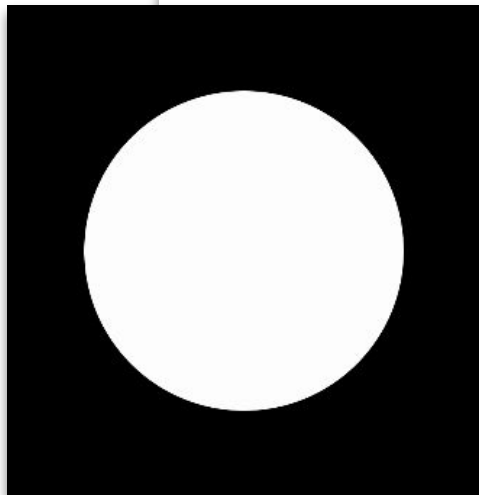
A GUI for config manipulation

- Features:
 - Adding & deleting hotspots
 - Text fields for hotspot title and description, option to import from file
 - Managing associated media files (images & videos)
 - Importing/Exporting of the config file for portability



Frontend development - Beta release

- Info display improvements
 - Created a dynamic layouting system so that it's easier to add new layouts in the future
 - Layouts self declare the media they are compatible with
- Fully animated hotspot projections



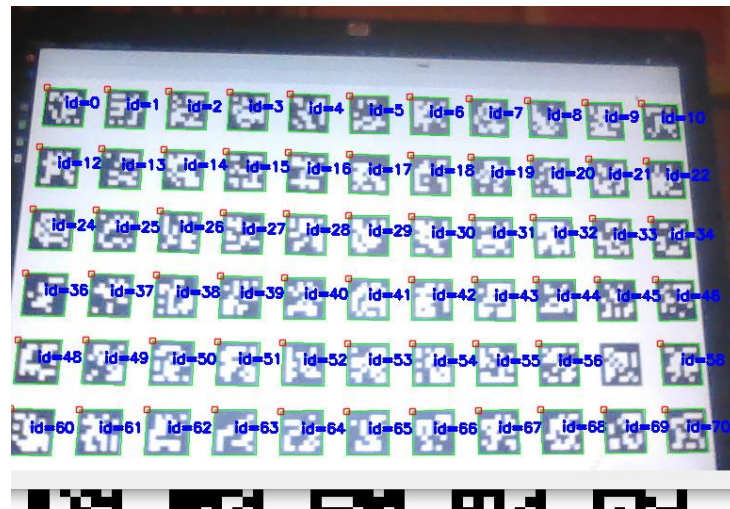
Hotspot 3

This is an amulet from Ancient Egypt, it is made of a rare jade material and is meant to resemble an Egyptian god.



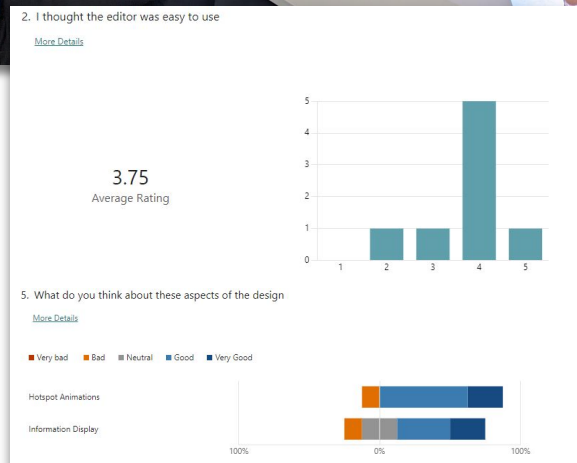
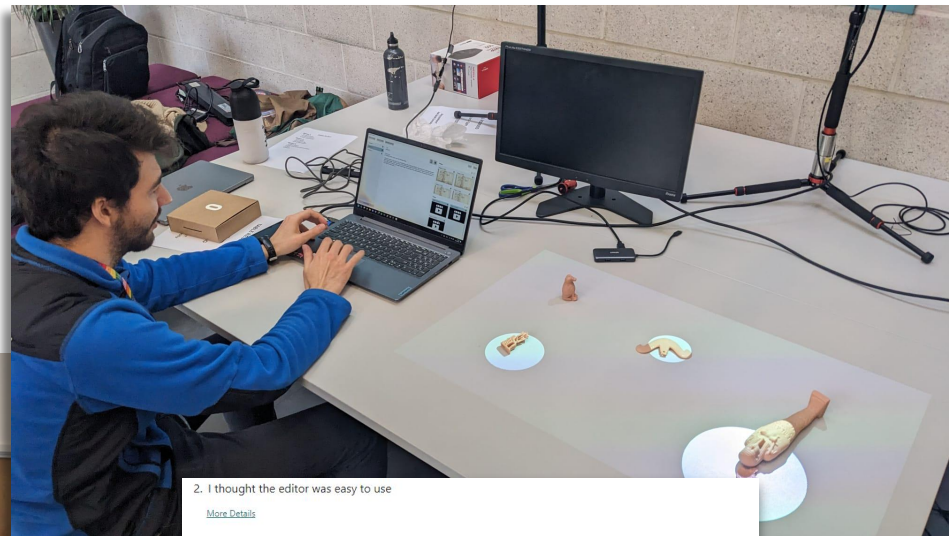
Backend development - Beta release

- Hand landmarking
 - Developed hand landmarking into a properly written system
- Calibration
 - Exploring: manual calibration; ArUco markers; ML-driven object detection
 - Settled on ArUco markers with openCV and developed it further
- Integration with frontend (argh)
 - 2 exported functions
 - Each sends data back to the frontend
 - Converting types between Python and C#



User testing

Photos and data from live demo
on 15.03.2024



Frontend development - Final release

- UI overhaul
 - Reworked styles for the application based on Microsoft's Fluent Design guidelines
 - More intuitive layouts based on user feedback from live demo
- New hotspot animations
 - More eye-catching and faster, based on feedback from users and the client
- Quality of Life changes
 - Fixed numerous bugs found during live demo
 - Added application icon, splash screen, information dialogs, camera chooser and more...
- Improved robustness
 - Error handling system to display errors to users
 - More thorough error detection for components of the system (file handler, Python backend, ...)

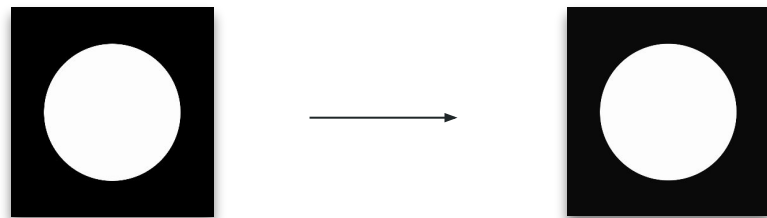
Changes from user testing to final release

Hotspot animations - Make them more obvious

"More obvious animation or visual clues"

"Have a more clear change when a hotspot has been selected"

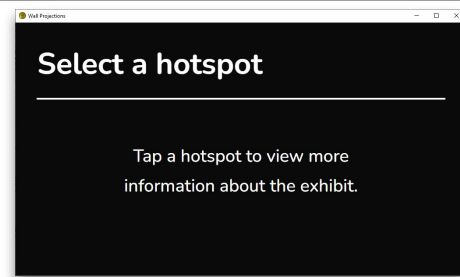
"More visual interest in terms of hotspot design"



Interaction - Prompt to tell users to interact

"Add some instructions so people know how to start interacting"

"Some kind of prompt telling users to point to the artefacts"

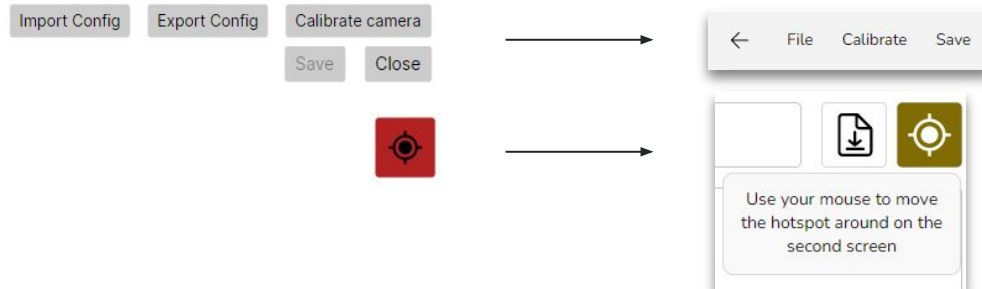


Editor - Make more intuitive

"The editing of hotspot locations was significantly unintuitive"

"more instruction may help the process flow better."

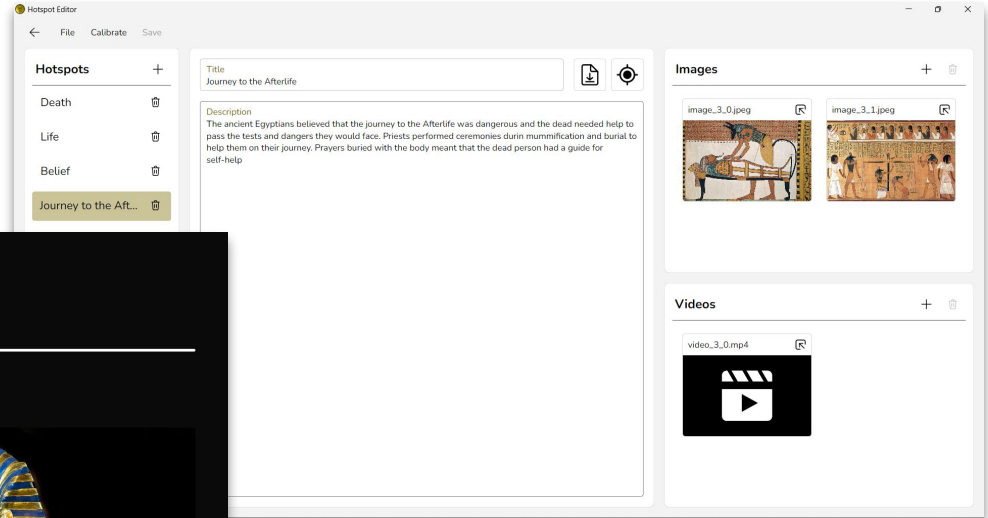
"Didn't feel super intuitive"



Frontend development - Final release

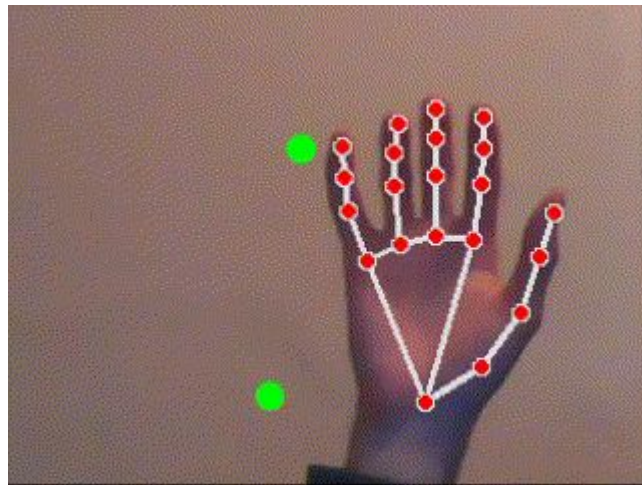
Death

The ancient Egyptians identified the dead with the god Osiris, who had been murdered and brought back to life. They believed that the dead, could, like the god, live again. Ideas and beliefs about what happened after death changed over the 6,000 years of ancient Egyptian history. What did not change was the idea that people had to make special preparations in their lifetime for their burial.



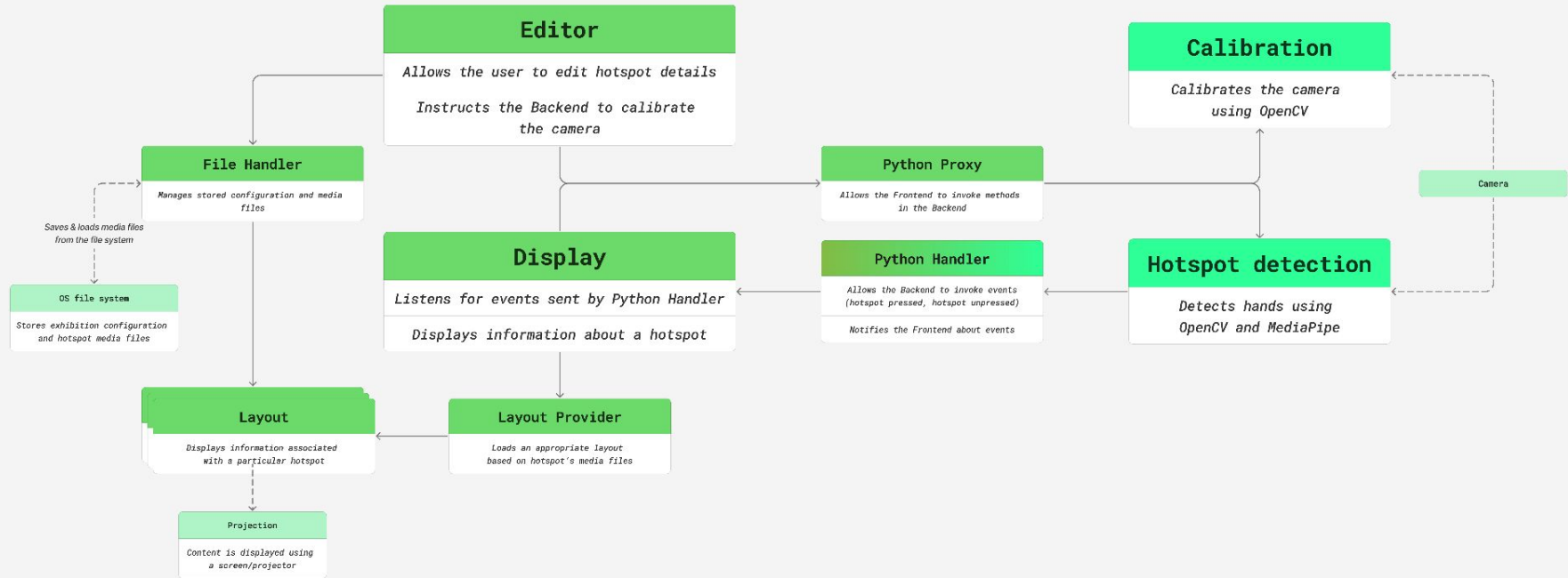
Backend development - Final release

- Various minor improvements
 - Embedded Python installation
 - Encapsulate video capture and move it to a background thread
 - Improve robustness with defensive programming
 - Camera identification
- Unit testing
 - Dependency injection to make tests viable
 - Test the hotspot detection against a pre-recorded video
 - Continuous integration



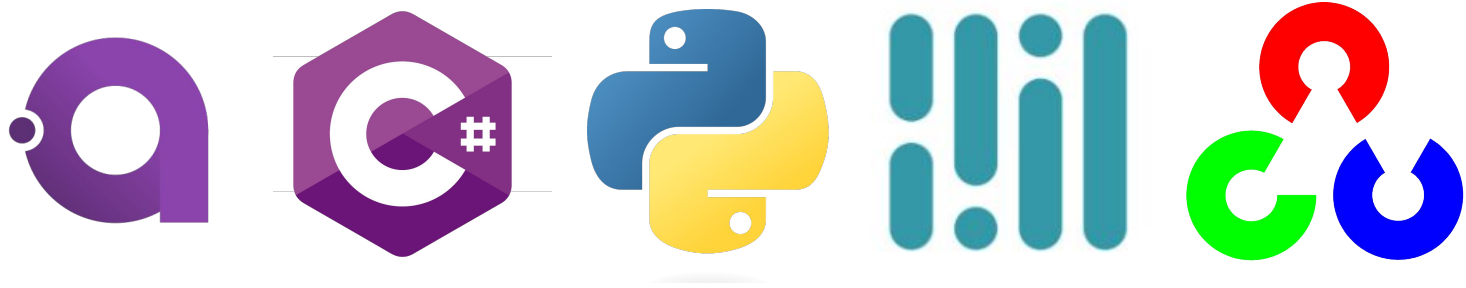
High-level System Architecture

Web Projections - Final



Technology Stack

Our project is written in 2 languages: *C#* for the Frontend and *Python* for the Backend. The former is built using *Avalonia*, a UI framework for *.Net* platform, and *python.net* library to communicate with the Backend. The latter uses *OpenCV* to calibrate the camera, and Google's *MediaPipe* to detect hands on the camera feed.



Computer Vision Hand Tracking

Record video of user interaction.



Downscale image.



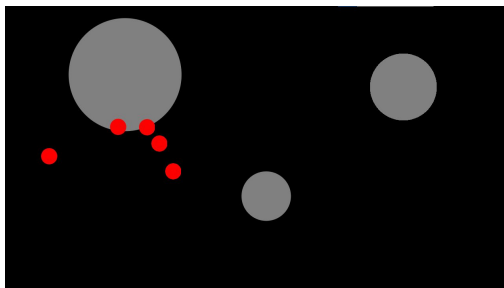
Run hand tracking model (Google's MediaPipe) to identify landmarks.



Select for just fingertips.



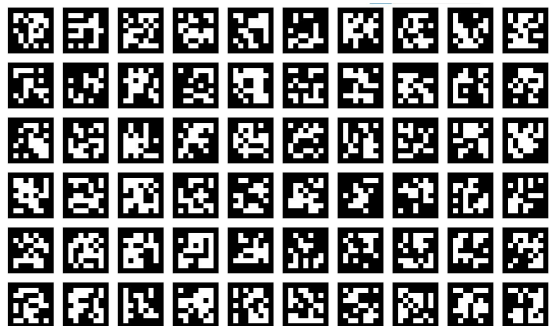
Use the Transformation Matrix to transform fingertips to "projector space".



Check if fingertips are inside hotspots, communicate with frontend.

Computer Vision Calibration

Project a grid of ArUcos with known coordinates in “projector space”.



Detect coordinates of ArUcos in “camera space” using OpenCV.

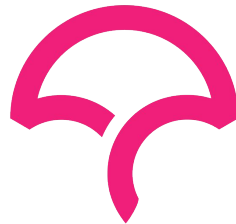


Using the two sets of coordinates generate a Transformation Matrix from “camera space” to “projector space”.

$$\begin{bmatrix} 2.26026357 & 0.0140671786 & -174.477112 \\ 0.105387057 & 2.14734851 & -192.239216 \\ 0.000143228792 & 0.0000940191141 & 1 \end{bmatrix}$$

CI/CD and Project management

- Weekly meetings to discuss project's trajectory and work on problems encountered during the week
 - After each major release our meeting was mostly focused on evaluation and planning next steps
- Client meetings on every major release
- A 'branch-per-feature' strategy for source control
- CI/CD using GitHub Actions
 - Quick unit testing on every push
 - Comprehensive testing for all platforms on every PR
 - Code coverage monitoring on every PR using *Codecov*
 - Automatic build and publish on every merge to *dev*
 - Automatic documentation website update on every push to *docs*
- Separate branch specifically for documentation



Problems

- LibVLCSharp - cross-platform embeddable video player library
 - Major flaw in the library - managed to find a workaround
 - Lack of availability on Apple Silicon macOS - the x64 binary can be run with some caveats.
- C# ↔ Python interop
 - Few options to choose from
 - Poor documentation
 - A very “clanky” solution overall
- Raspberry Pi
 - Initial OS setup
 - Downloading all required dependencies
 - Insufficient GPU resources

Final evaluation

- The project exceeds expectations
 - We were tasked to develop a proof-of-concept, but we have made a production-ready system
 - It's easy to setup with limited technical knowhow, Calibration allows a variety of setups.
- What we have learnt
 - How to design a fairly large system in a maintainable way
 - How to design a good looking and feeling application
 - Loads about practical computer vision and ML models
 - Playing videos on multiple platforms is awful
 - Detecting available cameras on multiple platforms is awful
- What we could have done differently
 - Use C++ bindings for *MediaPipe* & *OpenCV* directly instead of Python wrappers because integration is a PAIN
 - Experiment with more performant solutions for Raspberry Pi support



Questions